

Edge cases (empty vs. missing) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Where this exercise is saved: `practice/env_config_practice.py`, under its `# Edge cases` section — the same file also holds the [Intermediate exercise](#), under an `# Intermediate` section. Run it with `cd practice && python env_config_practice.py`.

Builds on: the `# Intermediate` section of that same file — you are testing *that* loader against the two tricky `.env` states, so keep both sections in the one file. The `MissingConfigError` used below is the same class the [Build Task](#) ends up with in `practice/build_task/exceptions.py`.

Story — `env_config_practice.py` (Edge cases section): "missing" isn't one thing — a key that was never set and a key set to an empty string are two different states that look the same to a careless check. This exercise decides, on purpose, which one a required secret should tolerate. **If not:** `require_env()` in the Build Task would ship with an untested assumption about empty strings, and a genuinely blank API key could sail through as if it were valid.

Basic Version

Approach 1 — the strict, direct way

```
# practice/env_config_practice.py – Edge cases section
import os

class MissingConfigError(Exception):
    pass

def require_env(key):
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            "Required environment variable is missing: " + key
        )
    return value

# case 1: simulate an empty .env file -- the key was never set
```

```
os.environ.pop("OPENAI_API_KEY", None)
try:
    require_env("OPENAI_API_KEY")
except MissingConfigError as e:
    print("Caught it:", e)

# case 2: simulate .env containing "OPENAI_API_KEY=" -- present, but empty
os.environ["OPENAI_API_KEY"] = ""
try:
    require_env("OPENAI_API_KEY")
except MissingConfigError as e:
    print("Caught it:", e)
```

Expected output:

```
Caught it: Required environment variable is missing: OPENAI_API_KEY
Caught it: Required environment variable is missing: OPENAI_API_KEY
```

Both edge cases raise the same error here — that's the strict choice, and it's the right default for a secret like an API key. This version works correctly. It's missing type hints, which is fine for a first working version.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — strict: empty string counts as missing

```
# practice/env_config_practice.py – Edge cases section
import os

class MissingConfigError(Exception):
    pass

def require_env(key: str) -> str:
    # why: strict on purpose – an API key that's set but empty is never
    # usable, so it's treated the same as not being set at all.
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

# case 1: key was never set at all
# how: pop(..., None) removes it if present, does nothing if not – never raises
os.environ.pop("OPENAI_API_KEY", None)
try:
    require_env("OPENAI_API_KEY")
except MissingConfigError as e:
    print(f"Case 1 - not set at all: {e}")

# case 2: key is set, but empty
# when: simulates a .env line like "OPENAI_API_KEY=" with nothing after the "="
os.environ["OPENAI_API_KEY"] = ""
try:
    require_env("OPENAI_API_KEY")
except MissingConfigError as e:
    print(f"Case 2 - set but empty: {e}")
```

Expected output:

Case 1 - not set at all: Required environment variable is missing:
 OPENAI_API_KEY
 Case 2 - set but empty: Required environment variable is missing: OPENAI_API_KEY

Approach 2 — lenient: only `None` counts as missing

```
# practice/env_config_practice.py – Edge cases section
import os

class MissingConfigError(Exception):
    pass

def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None:
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

# case 1: key was never set at all
os.environ.pop("OPENAI_API_KEY", None)
try:
    require_env("OPENAI_API_KEY")
except MissingConfigError as e:
    print(f"Case 1 - not set at all: {e}")

# case 2: key is set, but empty
os.environ["OPENAI_API_KEY"] = ""
result = require_env("OPENAI_API_KEY")
print(f"Case 2 - set but empty, returned: {result!r}")
```

Expected output:

Case 1 - not set at all: Required environment variable is missing:
 OPENAI_API_KEY
 Case 2 - set but empty, returned: ''

Difference from Basic: both Intermediate approaches add full type hints (`key: str` , `-> str`), and both test the two edge cases directly with real, side-by-side code instead of describing them in words. The real difference is between the approaches themselves: Approach 1 raises in case 2,

Approach 2 doesn't — it returns the empty string instead. That's a genuinely different behavior, not just a style choice, and which one is "right" depends on whether an empty value is ever a valid setting for that particular key.

Which one should you actually write? Intermediate Approach 1 (strict, typed) is what you should default to — an API key that's empty is never usable, so treat it as missing.